sfizz for MPC OS

User manual · version 1.1.0



The sfizz 1.2.3 SFZ sampler as a native instrument plugin for MPC OS hardware, Gen1 and Gen2, on a one-page screen modelled on sfizz's own editor. Play SFZ libraries straight from SD cards and USB drives, streamed from disk, with key switches, the instrument's own controls on knobs, Scala tunings and high-quality interpolation.

Contents

1. Requirements
2. Installing and uninstalling
3. Quick start
4. The screen
5. Loading instruments
6. Controls and MIDI
7. Settings reference
8. Tuning
9. Projects
10. Performance and sound quality
11. Troubleshooting
12. Credits and licences

sfizz is by the sfztools contributors; this MPC port is a community project, not an official sfztools release, and is not affiliated with inMusic or Akai. MPC and Force are trademarks of their owners.

1. Requirements

- **An MPC OS device**, either generation, each with its own zip:

Generation	Devices	Zip
Gen1 (RK3288, 32-bit)	Force, MPC Live, Live II, One, X, Key 61	`sfizz-1.1.0-mpc-armv7.zip`
Gen2 (RK3588, 64-bit)	MPC Live III and other RK3588 units	`sfizz-1.1.0-mpc-aarch64.zip`

  The installer checks that the zip matches the device and names the right one if it doesn't. Both builds have been tested on hardware (an Akai Force and an MPC Live III, MPC OS 3.9.1).
- **MPC OS 3.x.** The screen uses the 3.x skin format.
- **Root shell access (SSH)** to the device. Stock MPC OS doesn't offer this: the device must be modded. The installer edits MPC's settings file to register the plugin.
- **SFZ instruments or SoundFonts** (`.sf2`) on an SD card, a USB drive or in the plugin's own library folder. sfizz reads WAV, FLAC, Ogg Vorbis, AIFF and WavPack samples.

> Installing plugins this way is unofficial. Back up your projects and settings first, and use it at your own risk.

2. Installing and uninstalling

Scripted install (recommended)

1. Download the zip for your device (`sfizz-1.1.0-mpc-armv7.zip` for Gen1, `sfizz-1.1.0-mpc-aarch64.zip` for Gen2) from the project's GitHub Releases page and unzip it on your computer. Both unzip to a folder named `sfizz-1.1.0` .
2. Copy the whole folder to the device:

   ```
   scp -r sfizz-1.1.0 root@<device-ip>:/tmp/
   ```

3. Run the installer:

   ```
   ssh root@<device-ip> sh /tmp/sfizz-1.1.0/install.sh
   ```

The installer checks the device, asks for confirmation, **stops MPC** (save your project first), copies the plugin folder to `/sdcard/Synths/sfztools - VST - sfizz/` , backs up `MPC.settings` , adds sfizz to MPC's plugin list and starts MPC again. `-y` skips the question; `-t <folder>` installs into another Synths folder (for example on a card; on Gen2 an internal Synths folder may be read-only, so a card or drive's `Synths` folder may be needed). Running it again upgrades in place. The MPC OS plugin catalog's installers (the `mpc-store.sh` command and the desktop app) pick the right zip for the device by themselves.

Standalone install with vstscanner (Gen1)

If your device registers plugins with `vstscanner.sh` from the Force VST plugins distribution (for example on MockbaMod), use `sfizz-for-MPC-OS-1.1.0.zip` instead: the same Gen1 plugin as a plain folder. Example for a MockbaMod card mounted at `/media/662522` :

1. Copy the `sfztools - VST - sfizz` folder into `/media/662522/Synths` , next to `vstscanner.sh` .
2. Point the Instruments tile's Default preset at that folder:

```
ssh <device-ip> "sed -i 's|%payload-path%|/media/662522/Synths|' \
  '/media/662522/Synths/sfztools - VST - sfizz/Presets/0000-Default.xpl'"
```

3. Run the scanner:

   ```
   ssh <device-ip> 'sh /media/662522/Synths/vstscanner.sh'
   ```

   (afterwards just `vstscanner`). MPC restarts; sfizz is listed under manufacturer **sfztools**.

The zip's `README.md` covers other firmware (another drive, passing the settings file to the scanner). Use one install method per device: the scanner rebuilds MPC's whole plugin list from the folders it finds, so don't mix it with `install.sh`. To update, replace the folder (keep files you put in `sfizz/Instruments` and `sfizz/Scales`), repeat step 2 and run the scanner.

Manual install

The zip's `INSTALL.md` lists the manual steps: copy the plugin folder, stop MPC, back up and edit `MPC.settings`, start MPC.

Uninstall

```
ssh root@<device-ip> sh /tmp/sfizz-1.1.0/uninstall.sh
```

This removes the plugin folder and its plugin-list entry, and restarts MPC. SFZ files you put in the plugin's `sfizz/Instruments` folder are kept. Projects that used sfizz load without it.

3. Quick start

1. Copy an SFZ instrument folder to an SD card or USB drive (keep the `.sfz` file next to its samples folder) and insert it. A good free one: *Salamander Grand Piano V3* (github.com/sfzinstruments/SalamanderGrandPiano).
2. On a track, add **sfizz** from the plugin browser (Instrument plugins), or from Sounds › INSTRUMENTS, where it has its own sfizz tile: tap it, then **Default**. A new sfizz plays a simple sine.
3. Tap the ⌄ button at the top right of the file box. The browser opens.
4. Tap your card, then folders, then the `.sfz` file. The name, counts and memory update when it has loaded.
5. Play. Turn the Q-Links: they control Volume, Pan and the instrument's own controls.
6. Tap ⌄ again to close the browser.

4. The screen

sfizz's desktop editor has four tabs. On the MPC they are side by side on one page: the header, then **Information**, **Controls** and **Settings**, then the piano.

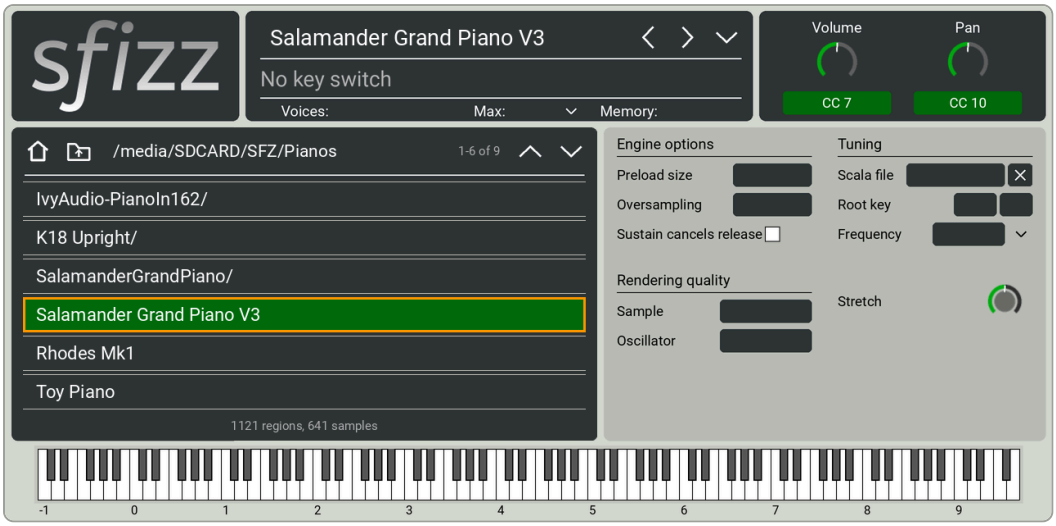


The screen with Salamander Grand Piano V3 loaded.

1. **File name** of the loaded instrument (without `.sfz`). Shows *Loading…* while loading.
2. ‹ › load the previous / next SFZ file in the same folder.
3. ⌄ opens and closes the SFZ browser.
4. **Key switch**: the current key switch's note (badge) and name. Instruments without key switches show *No key switch*.
5. **Voices** sounding now · **Max** voices (tap to choose) · **Memory** used by sample data in RAM.
6. **Volume** and **Pan**: MIDI CC 7 and CC 10 for the instrument (0–127; pan 64 is centre).
7. **Information**: the instrument's curves, masters, groups, regions and samples.
8. **Controls**: up to eight of the instrument's own controllers, named as the instrument names them, with their CC number.
9. **Settings**: engine options, rendering quality, tuning and playing mode (section 7).
10. **Piano**: all 128 notes and their octave numbers (middle C is C4). A key shows pressed (white keys grey, black keys lighter, as in sfizz) while its note is held, from pads, a MIDI controller or the sequencer. It is a display: touching it doesn't play.

> Not on the MPC screen, compared with the desktop editor: the output meters, the Theme and Zoom options, the "when freewheeling" quality settings (MPC never freewheels) and the file menu's Edit, Create and Open folder entries.

5. Loading instruments



The browser covers the Information and Controls panels while it is open.

The browser

- ⌂ (home) lists the places: **Plugin Library** (the plugin's own `sfizz/Instruments` folder), every SD card and USB drive, and `sdcard` (the internal storage).
- Tap a folder (shown with a trailing `/`) to open it, and the **folder-up** button to go back. A tap acts once: a second touch within a third of a second of a folder opening is ignored, so one tap never opens two things.
- Tap an SFZ file to load it. The loaded file is lit green. The browser stays open so you can try others.
- ∧ ∨ page through long folders, six rows at a time; the counter shows where you are.
- The bottom line reports the result: the number of regions and samples, or why something didn't load.

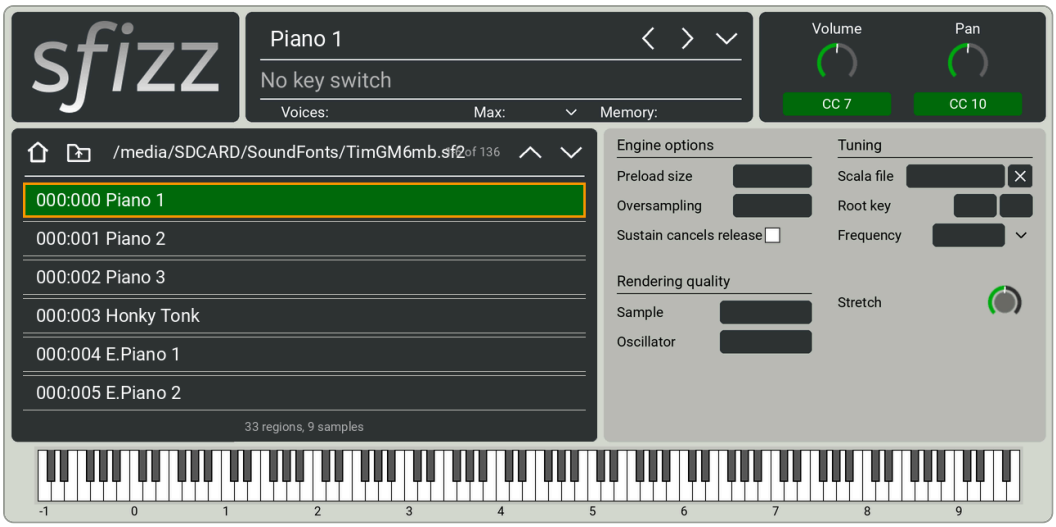
Where to put instruments

- **On a card or drive**: any folder. Keep each instrument as its author ships it: the `.sfz` file finds its samples by their path relative to it.
- **In the plugin library**: `/sdcard/Synths/sfztools - VST - sfizz/sfizz/Instruments/`. The installer keeps this folder when you upgrade or uninstall.

Loading takes a moment

sfizz reads the start of every sample into memory (the *preload*) and streams the rest from disk while you play. A large library can take several seconds to load from a card. The plugin is silent while it loads; notes played during a long load are skipped.

SoundFonts (.sf2)



A SoundFont opened in the browser: its presets, numbered bank:program.

- **Open a SoundFont** like a folder: tap the `.sf2` file (it shows with a slash) and the browser lists its presets as *bank:program name*, for example `000:000 Piano 1`; General MIDI drum kits are in bank 128.
- **Tap a preset** to load it. The first time, sfizz converts it into an SFZ instrument and WAV samples in the plugin's `sfizz/SF2 Cache` folder: the status line says *Converting…*, which takes a moment for big presets, and the instrument you had keeps playing until it's done. Later, the same preset loads at once.
- ‹ › in the file box step through the SoundFont's presets.
- **Program change**: while a SoundFont preset is loaded, a MIDI program change (bank from CC 0) switches to that preset of the same SoundFont, as SoundFont players do, if MPC sends program changes to the plugin.
- A project saves the SoundFont and the preset, and finds the file by name if it moved.
- The cache keeps up to 1 GB of converted samples, dropping the least recently used SoundFonts first. Deleting `SF2 Cache` is always safe: presets are converted again when needed.

The conversion follows the SoundFont 2.04 rules: key and velocity splits, loops, root key and tuning, attenuation, pan, volume and modulation envelopes, filter, LFOs and exclusive classes, and 24-bit samples. Custom SF2 modulators beyond the standard ones are not converted, so a few presets that rely on them sound plainer than in their editor. Compressed SF3 files can't be read: use the SF2 version of the SoundFont.

Mono and Legato

The **Playing** section turns any instrument, SFZ or SoundFont, into a monophonic one: **Mono** retriggers each note, **Legato** slides the sounding note to the new pitch without a new attack, and **Glide** sets how long the slide takes. Legato bends the sample that is already playing, so very large intervals sound stretched, as on hardware samplers; instruments that come with their own legato samples (SFZ `trigger=legato`) do their legato in Poly mode, as their author made it. sfizz itself has no mono mode or portamento: these are this port's.

Instruments that use the extended SFZ format

sfizz supports SFZ v1 and v2 and many ARIA extensions (key switches, CC curves, release triggers, crossfades, effects). Opcodes it doesn't support are ignored, as on the desktop.

6. Controls and MIDI

Q-Links

Bank	Q-Links
Bank 1	Volume, Pan, Controls 1–6
Bank 2	Controls 7–8, Frequency, Stretch, Sample quality, Oscillator quality, Mode, Glide

A Q-Link on an empty Controls slot does nothing. Settings that reload samples (Max voices, Oversampling, Preload size) are not on Q-Links, so a turn can't interrupt playing.

The Controls panel

sfizz lists the MIDI controllers the instrument uses. The panel shows up to eight of them: first those the instrument names (its `label_cc` lines), then the others by number. CC 7 and CC 10 are left out because they are the Volume and Pan knobs. A controller that doesn't fit still works from MIDI.

MIDI

- **Every controller reaches the instrument**, including CC 20–35 (which some MPC plugins keep for Q-Links). Knobs follow the controllers they show.
- Pitch bend, channel and polyphonic aftertouch, and the sustain pedal (CC 64) go to sfizz.
- All Sound Off (CC 120) and All Notes Off (CC 123) silence the instrument.
- The plugin plays on any MIDI channel the track sends.
- Program change (with bank select, CC 0) picks a preset of a loaded SoundFont; SFZ instruments ignore it.

7. Settings reference

Tap a dark value button to open its list of options; tap an option to pick it and close the list. The lists open large for the touchscreen, below or above their button, in columns when they are long.

Setting	Values (default in bold)	What it does
Max (voices)	1, 2, 4, 8, 16, 32, **64** (Gen1), **96** (Gen2), 128, 160, 192, 224, 256	How many voices can sound at once. Lower it if the device runs out of CPU with dense playing.
Volume	0–127 (**the instrument's default**, often 100)	MIDI CC 7 to the instrument.
Pan	0–127 (**64**, centre)	MIDI CC 10 to the instrument.
Preload size	4, 8, 16, **32**, 64, 128, 256 kB	How much of each sample sfizz keeps in memory; the rest streams from disk. Raise it if a slow card makes note starts drop out; lower it to save memory. Changing it reloads the samples.
Oversampling	**1x**, 2x, 4x, 8x	Upsamples the samples when they load. It can clean up heavily pitched sounds, but multiplies memory and load time. Changing it reloads the samples.
Sustain cancels release	**Off**, On	sfizz's option of the same name: decides whether holding the sustain pedal cancels release samples, as some other SFZ players do.
Sample (quality)	Nearest, **Linear** (Gen1), Polynomial, **Sinc 8** (Gen2), Sinc 12, 16, 24, 36, 48, 60, 72	How samples are resampled when played at another pitch or sample rate. Higher is cleaner and costs more CPU (section 10).
Oscillator (quality)	Nearest, **Linear** (Gen1), **High** (Gen2), Dual-High	The same for sfizz's built-in waveforms (`*sine` , `*saw` and wavetables).
Scala file	**DefaultScale** (equal temperament), files in `sfizz/Scales`	Tap to step to the next scale; × goes back to equal temperament.
Root key	note **C** · octave **4**	The key the scale starts on (C4 = MIDI note 60).
Frequency	300–500 Hz (**440.0**)	Tuning of A4. Drag the value or turn its Q-Link; the ⌄ next to it picks a historical pitch (Baroque 415, French Law 435, European 442 …).
Stretch	0–100% (**0**)	Piano-style stretch tuning: octaves widen towards the ends of the keyboard.

Mode (Playing)	**Poly**, Mono, Legato	**Poly**: every note sounds, as the instrument is written. **Mono**: one note at a time; a new note cuts the previous one and plays its own attack. **Legato**: one note at a time, and a note played while another is held doesn't restart the sound: the note already playing changes pitch, keeping its attack and tone. In both, letting go of the newest note goes back to the one still held (last-note priority). Key-switch keys keep working as usual.
Glide	**Off** to 2 s	Portamento for Mono and Legato: the pitch slides from one note to the next over this time (Off: it jumps). Drag the value or turn its Q-Link. No effect in Poly.

8. Tuning

sfizz tunes with Scala (`.scl`) files, a common format for microtonal and historical scales; thousands are freely available online.

1. Copy `.scl` files to `/sdcard/Synths/sfztools - VST - sfizz/sfizz/Scales/`.
2. Tap the **Scala file** button to step through them (equal temperament first). Tap **×** to reset.
3. Set the **Root key** (note and octave) the scale is built on.
4. Set the reference pitch with **Frequency**, or pick a historical one from its ⌄ list.

The new scale list is read when you step, so files copied while MPC runs show up without a restart.

9. Projects

A saved project keeps, for each sfizz instance: the instrument file, the Scala file, every setting, and the positions of the Volume, Pan and Controls knobs. When the project loads, the instrument loads again in the background.

- Instruments are saved by their full path. If the file has moved (another card, a renamed drive), sfizz looks for a file of the same name in the plugin library, on every card and drive, and in `/sdcard`, up to four folders deep, and loads the first it finds.
- If it isn't found, the file name shows *(missing)* and the browser's status line says so. Load it again from the browser and save the project.

10. Performance and sound quality

The MPC's processor is shared by every track and plugin. What costs CPU in sfizz, from most to least:

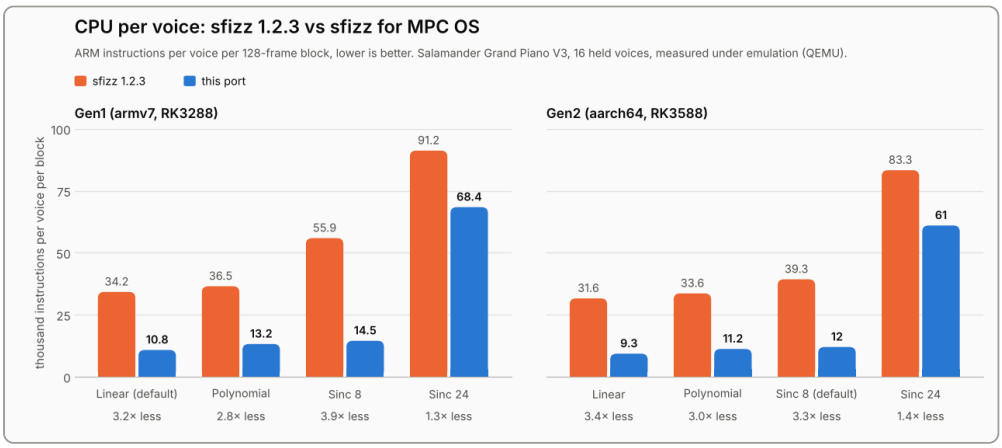
- **Voices sounding**: sustained piano chords with the pedal down can use dozens. Lower **Max** if needed.
- **Sample quality**: per voice, Sinc 8 costs about 10% more than Polynomial, Sinc 24 nearly 5× Polynomial, and the longer filters more still.
- **Oversampling**: more data per voice.

Loading samples while you play. Desktop sfizz reads the whole rest of a sample file the moment a key is hit, on a high-priority thread, and keeps it in memory with no limit; on the MPC that briefly maxed out the processor on every note with large libraries, and long sessions of chords could run out of memory. This port reads files gradually instead, a little ahead of what is playing, at normal priority, within a memory budget (a third of the memory free when the plugin starts, between 48 and 384 MB on Gen1 and up to 1 GB on Gen2). Files no note uses are released after a few seconds. If the budget runs out (very many long notes at once), the notes that have played longest fade out quickly, as when sfizz runs out of voices, so new notes always sound. The sound is otherwise exactly the same.

Gen1. Sample and Oscillator quality start at **Linear**, the lightest setting that still interpolates, so a new sfizz leaves the most CPU for everything else. When there's room, raise Sample quality: Polynomial (sfizz's desktop default) or Sinc 8 are cleaner on samples played far from their original pitch (in offline measurements Sinc 8 removed about 24 dB of interpolation distortion over Polynomial on bright samples pitched down). Samples played near their own pitch, as in most multisampled pianos, sound nearly the same at any setting. sfizz keeps samples in 32-bit float, so 24-bit libraries keep their full depth. A project keeps the quality it was saved with.

Gen2. The RK3588 has several times the processing power and memory, so a new sfizz there starts at **Sinc 8** sample quality, **High** oscillator quality and **96** voices, keeps up to 1 GB of decoded samples and reads 2 seconds ahead of each note (Gen1: 1.25 s), which gives slow cards and drives more slack. In offline measurements Sinc 8 costs about 1.3 times Linear's CPU per voice on Gen2, still about a third of what desktop sfizz needs on the same processor. A project made on one generation opens on the other with its own settings.

> If you hear crackles: lower Sample quality (Linear is the lightest), lower Max voices, or check that other plugins on the project aren't already using most of the CPU. If note starts drop out on a slow card, raise Preload size.



Work per voice, desktop sfizz 1.2.3 against this port, on both generations (measured offline under emulation; lower is better).

11. Troubleshooting

Problem	What to do
sfizz isn't in the plugin browser	Run `install.sh` again and let MPC restart. Check the zip matches the device (armv7 for Gen1, aarch64 for Gen2) and that it runs MPC OS 3.x.
The installer refuses the zip	It is the other generation's build: download the zip it names.
The screen is empty or wrong	MPC OS 2.x can't read this skin: update to 3.x.
An instrument loads but some notes are silent	Samples are missing: copy the whole instrument folder, keeping its layout. Check that sample file names match the `.sfz` exactly.
"Could not load …"	The `.sfz` couldn't be read or parsed. Try it in sfizz on a computer.
"No SFZ files in …" after ‹ ›	The folder has no `.sfz` files at its top level; open a subfolder with the browser.
A card doesn't appear in the browser	Insert it before opening the browser, then tap ⌂ to refresh the list.
A Controls knob is missing	The panel shows eight controllers; the others still work from MIDI.
Silence for a moment after changing Max, Oversampling or Preload size	Expected: these reload the engine.
Long held notes end early while playing many keys with a big library	The sample memory budget is full (it is set from the memory free when the plugin starts): the oldest notes give way to new ones. Close other memory-hungry plugins or unload unused project samples, then reload the project; or lower Max voices.
"Can't read … .sf2"	The file isn't a SoundFont 2, is damaged, or is a compressed SF3. Use the SF2 version.
"Could not convert …"	Usually the card holding the plugin is full: free some space (or delete old folders in `sfizz/SF2 Cache`) and tap the preset again.

12. Credits and licences

- **sfizz** by the sfztools contributors, BSD-2-Clause (github.com/sfztools/sfizz). This port uses the 1.2.3 engine with one change, to how it reads sample files (the sound is unchanged); its libraries keep their own licences.
- **VST2 wrapper, skin tools and installer**: mpc-vst-plugins by sd88me and contributors, MIT.
- **Artwork**: drawn after sfizz's editor; the sfizz logo and emblem come from sfizz-ui (BSD-2-Clause). Roboto, Apache-2.0.
- **This port**: BSD-2-Clause. Source: github.com/gmorb/mpc-vst-sfizz.